

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – REGEX PATTERN MATCHING ACTIVITY

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO3	Assessment:	Tool 1 – Regex Pattern Matching Activity
Weightage:		Total Marks:	25 Marks
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL3 and BL4).			
Student Name:	P.Sunayana	Reg. No.:	192424303
Section / Batch:	Slot A - 2024	Date of Submission:	12-08-2026
Faculty In-charge:	K. Veena devi	Project Mode:	Individual Submission

Code of Conduct

I _P.Sunayana(192424303)_(Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: ____P.Sunayana____

Understanding (5 Marks)	Pattern Design (5 Marks)	Data Extraction and Validation (5 Marks)	Explanation (5 Marks)	Applications (5 Marks)	Total (25 Marks)

Set - 01

1. Break down the regex `\d{3}-\d{2}-\d{4}` token by token. What does each component match, and what real-world data format does this pattern describe?

Consider quantifiers, character classes, and literal characters separately.

Break down the regex `\d{3}-\d{2}-\d{4}` token by token

The regular expression is:

`\d{3}-\d{2}-\d{4}`

This pattern consists of character classes, quantifiers, and literal characters.

Token-by-token explanation

Token	Type	Meaning
<code>\d</code>	Character class	Matches one digit, normally 0–9
<code>{3}</code>	Quantifier	Requires exactly 3 occurrences
<code>-</code>	Literal	Matches a hyphen character
<code>\d</code>	Character class	Matches one digit
<code>{2}</code>	Quantifier	Requires exactly 2 digits
<code>-</code>	Literal	Matches a hyphen
<code>\d</code>	Character class	Matches one digit
<code>{4}</code>	Quantifier	Requires exactly 4 digits

How the engine processes it

For the input:

123-45-6789

the matching occurs as:

123 - 45 - 6789

^^^ ^^ ^^^^

- `\d{3}` → 123
- `-` → -
- `\d{2}` → 45
- `-` → -
- `\d{4}` → 6789

Therefore, the complete match is:

123-45-6789

Real-world format

This pattern commonly represents a United States Social Security Number (SSN).

However, it only validates the syntactic structure. It does not verify whether the number is actually a valid or assigned SSN.

Conclusion

The regex means:

Match exactly 3 digits, followed by a hyphen, followed by 2 digits, another hyphen, and exactly 4 digits.

-
2. Given the pattern `^(https?|ftp):\\V[^\\s]+$`, identify which part handles protocol variation and explain how the alternation operator changes the set of matching strings.

Trace how `https?` differs from `https|http` in coverage.

Protocol variation in `^(https?|ftp):\\V[^\\s]+$`

The complete pattern is:

`^(https?|ftp):\\V[^\\s]+$`

This pattern is designed to recognize URLs using specific protocols.

Breaking the pattern down

Part	Meaning
<code>^</code>	Beginning of the string
<code>`(https? ftp)`</code>	<code>ftp`</code>
<code>:</code>	Literal colon
<code>\\V</code>	Literal <code>://</code>
<code>[^\\s]+</code>	One or more non-whitespace characters

Which part handles protocol variation?

The protocol variation is handled by:

`(https?|ftp)`

The parentheses create a group and the `|` symbol represents alternation, meaning "OR".

Therefore:

https?|ftp

means:

http OR https OR ftp

How does https? work?

The ? quantifier means that the preceding character is optional.

Therefore:

https?

can be interpreted as:

http + optional s

It matches:

http

https

It does **not** match:

htt

httpss

httpx

https? versus https|http

These two expressions have equivalent coverage:

https?

and:

https|http

Both match:

http

https

However, https? is more concise because it expresses the fact that only the final s is optional.

Effect of alternation

With:

(https?|ftp)

the complete set of protocols becomes:

http

https

ftp

Examples accepted by the complete regex include:

http://example.com

https://example.com

ftp://files.example.com

The `[^\s]+` portion ensures that at least one non-whitespace character occurs after `://`.

Conclusion

The alternation operator expands the set of possible matches by allowing one of several alternatives. Here, it allows **HTTP, HTTPS, or FTP**.

3. Compare greedy vs. lazy quantifiers using `<.*>` versus `<.*?>` on the string `<b>bold</b>`. Trace exactly what each engine captures and explain why the outputs differ.

Draw the backtracking steps for greedy matching.

Greedy vs. lazy quantifiers: `<.*>` and `<.*?>`

Consider the input:

`<b>bold</b>`

We compare:

`<.*>`

with:

`<.*?>`

Greedy matching: `<.*>`

The `*` quantifier is greedy by default.

It means:

Match zero or more characters, taking as many as possible while still allowing the rest of the pattern to succeed.

The engine sees:

`<b>bold</b>`

The `<` matches the first `<`.

Then:

.*

initially consumes as much as possible:

bold

The final > must now match.

Because the .* consumed the final > as well, the engine backtracks.

Backtracking steps

Conceptually:

Input: bold

Step 1:

bold

^

`>` required but no character remains

Step 2:

bold</

^

`>` required, but current character is `/`

Step 3:

bold

^

`>` required, but current character is `d`

Step 4:

bol

^

`>` required, but current character is `l`

...

Eventually:

`<b>bold</b>`

^

`>` matches

Therefore:

`<.*>`

matches the entire string:

`<b>bold</b>`

Lazy matching: `<.*?>`

The ? after * makes the quantifier lazy:

`.*?`

It means:

Match as few characters as possible while still allowing the complete pattern to succeed.

The engine starts:

`<`

Then `.*?` initially consumes nothing.

The final `>` is checked.

At the current position, the character is `b`, so `>` does not match.

The lazy quantifier expands by one character:

`<b`

Now the next character is `>`.

Therefore the final `>` matches.

The result is:

`<b>`

Comparison

Regex	Type	Result
<code><.*></code>	Greedy	<code>bold</code>
<code><.*?></code>	Lazy	<code></code>

Conclusion

Greedy quantifiers try to consume the maximum possible amount and then backtrack if necessary.

Lazy quantifiers try to consume the minimum possible amount and expand only when necessary.

4. Analyze why the pattern $(a^+)^+$ causes catastrophic backtracking on an input like `aaaaaX`. How does the nested quantifier structure create exponential match attempts?

Count the distinct ways the engine can partition the 'a' characters between the outer and inner groups.

Why $(a^+)^+$ causes catastrophic backtracking

The regex is:

$(a^+)^+$

Input:

`aaaaaX`

The pattern contains **nested quantifiers**:

$(a^+)^+$

↑ ↑

| |

| outer +

inner +

The inner a^+ can match one or more `as`.

The outer $^+$ can repeat the entire group.

Therefore, the same `a` characters can be divided into many possible groups.

For five `as`:

`aaaaa`

possible partitions include:

`aaaaa`

`a + aaaa`

`aa + aaa`

`aaa + aa`

`aaaa + a`

a + a + aaa

a + aa + aa

aa + a + aa

...

a + a + a + a + a

The number of possible ways to partition n characters into non-empty consecutive groups is:

$$2^{n-1}$$

For five characters:

$$2^4=16$$

Therefore, there are **16 possible partitions**.

For larger inputs, the number grows exponentially:

Number of as	Possible partitions
5	16
10	512
20	524,288
30	536,870,912

The final X cannot be matched because the regex expects only a characters.

Consequently, the engine repeatedly tries different partitions before concluding that the pattern fails.

Why it is catastrophic

The problem is not simply repetition. It is **ambiguous nested repetition**.

Both:

a+

and:

(a+)+

can represent the same characters in many different ways.

This creates an exponential number of possible matching paths.

Conclusion

(a+)+ is dangerous on failure cases because the engine may perform an enormous number of backtracking attempts. Such patterns should be avoided or rewritten when processing untrusted or large input.

5. Write a regex to extract all email addresses from an unstructured customer feedback log. What assumptions does your pattern make, and which valid email formats might it miss?

Think about subdomains, plus-addressing, and unusual TLDs.

Regex to extract email addresses from customer feedback

A practical email extraction regex is:

```
\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b
```

For example, given:

Customer said: "Please contact john.doe@example.com.

You can also reach support+india@example.co.in."

the regex extracts:

john.doe@example.com

support+india@example.co.in

Explanation

`\b`

Ensures a word boundary.

`[A-Za-z0-9._%+-]+`

Matches the local part before `@`.

Examples:

john

john.doe

support+india

user_name

The `+` quantifier requires one or more characters.

`@`

Matches the literal `@`.

[A-Za-z0-9.-]+

Matches the domain.

Examples:

gmail.com

mail.example.com

company.co.in

Finally:

\.[A-Za-z]{2,}

requires a dot followed by at least two alphabetic characters.

Assumptions

The pattern assumes:

1. The local part uses common characters.
2. The domain contains letters, numbers, dots, or hyphens.
3. The TLD contains at least two letters.
4. The email follows a conventional format.

It supports:

user@example.com

john.doe@example.com

user+test@example.com

user@mail.example.co.in

Possible limitations

It may not support every technically valid email address defined by the full email standards.

It may miss:

- Quoted local parts
- Internationalized email addresses
- Unusual special characters
- Some uncommon RFC-compliant formats

A complete RFC-compliant email regex would be extremely complicated. Therefore, a practical regex is usually preferable for customer feedback extraction.

6. A dataset contains dates in mixed formats: MM/DD/YYYY, DD-Mon-YYYY, and YYYY.MM.DD. Design a single regex with named capture groups that normalises all three formats into a structured output.

Use named groups like (?P<year>...) in Python.

Mixed date formats using named capture groups

The dataset contains:

MM/DD/YYYY

DD-Mon-YYYY

YYYY.MM.DD

Examples:

12/25/2026

25-Dec-2026

2026.12.25

In Python, a single regex can be constructed using alternatives:

```
import re
```

```
pattern = re.compile(
    r'(?:(?P<month1>\d{2})/(?P<day1>\d{2})/(?P<year1>\d{4})'
    r'|(?P<day2>\d{2})-(?P<month2>[A-Za-z]{3})-(?P<year2>\d{4})'
    r'|(?P<year3>\d{4})\.(?P<month3>\d{2})\.(?P<day3>\d{2}))'
)
```

First alternative

```
(?P<month1>\d{2})/(?P<day1>\d{2})/(?P<year1>\d{4})
```

Matches:

12/25/2026

and captures:

```
month1 = 12
```

```
day1 = 25
```

```
year1 = 2026
```

Second alternative

```
(?P<day2>\d{2})-(?P<month2>[A-Za-z]{3})-(?P<year2>\d{4})
```

Matches:

25-Dec-2026

and captures:

day2 = 25

month2 = Dec

year2 = 2026

Third alternative

`(?P<year3>\d{4})\.(?P<month3>\d{2})\.(?P<day3>\d{2})`

Matches:

2026.12.25

and captures:

year3 = 2026

month3 = 12

day3 = 25

Normalized representation

All three can then be converted into:

Year = 2026

Month = 12

Day = 25

Python does not allow the same named group to be defined multiple times in separate alternatives, so separate names such as year1, year2, and year3 are used and then normalized in code.

The regex validates the **format**, while Python's date functionality should be used to check whether the date actually exists.

-
7. You are preprocessing tweets for sentiment analysis. Write a pipeline of regex substitutions that removes URLs, mentions, hashtag symbols (but keeps the word), and repeating punctuation. Justify the order of operations.

Order matters — removing URLs before mentions avoids accidental partial matches.

Regex preprocessing pipeline for tweets

For sentiment analysis, the preprocessing pipeline can be:

```
import re
```

```
text = re.sub(r'https?://\S+|www\.\S+', '', text)
```

```
text = re.sub(r'@\w+', '', text)
```

```
text = re.sub(r'#(\w+)', r'\1', text)
```

```
text = re.sub(r'([!?,.])\1+', r'\1', text)
```

Step 1 – Remove URLs

```
https?://\S+|www\.\S+
```

This removes URLs such as:

```
https://example.com
```

```
http://news.com/article
```

```
www.example.com
```

Step 2 – Remove mentions

```
@\w+
```

removes:

```
@user
```

```
@company
```

```
@john123
```

Step 3 – Remove hashtag symbols but keep the word

```
#(\w+)
```

with replacement:

```
\1
```

converts:

```
#Python
```

into:

```
Python
```

Step 4 – Reduce repeated punctuation

```
([!?,.])\1+
```

converts:

```
Amazing!!!
```

Really???

Wow.....

approximately into:

Amazing!

Really?

Wow.

Why order matters

The order is important.

URLs should be removed first because they can contain characters that resemble mentions, punctuation, or other tokens.

Then mentions can be removed.

Hashtags are processed afterward because we want to preserve their semantic word.

Finally, repeated punctuation can be normalized.

8. Using lookaheads and lookbehinds, write a pattern that matches prices in a product catalog only when they appear inside a `<price>` tag and exceed \$99, without consuming the XML tags in the match.

Combine a lookbehind for the opening tag with a lookahead for the closing tag.

Matching prices inside `<price>` tags

Suppose the input is:

```
<price>$150</price>
```

A simple regex is:

```
(?<=<price>)\$([1-9]\d{2,})(?=</price>)
```

Lookbehind

```
(?<=<price>)
```

This is a positive lookbehind.

It means:

The current position must immediately follow `<price>`.

The opening tag is checked but not included in the match.

Price

`\$([1-9]\d{2,})`

- `\$` matches `$`.
- `[1-9]` requires the first digit to be from 1 to 9.
- `\d{2,}` requires at least two additional digits.

Therefore, it matches values such as:

\$100

\$150

\$999

\$5000

Lookahead

`(?=</price>)`

requires the price to be immediately followed by the closing tag.

It does not consume the closing tag.

Therefore:

`<price>$150</price>`

produces the match:

\$150

The XML tags are not part of the match.

Limitation

This simplified regex does not cover all monetary formats, such as:

\$99.99

\$1,000

\$100.50

Also, regex is not recommended for parsing arbitrary XML. An XML parser should be used for complex XML documents.

9. A colleague proposes using `.*` as the default pattern when the exact format is unknown. Critique this approach in the context of query processing — what correctness, performance, and maintainability risks does it introduce?

Think about false positives in SQL LIKE queries vs. full regex engines.

Critique of using `.*` as a default pattern

The pattern:

`.*`

means:

Match zero or more characters.

It is therefore extremely broad.

Correctness risk

Suppose we want to recognize a date.

Using:

`.*`

would accept:

hello

12345

abcxyz

even though none of these are dates.

Similarly, in query processing, a broad SQL pattern can generate many false positives.

For example:

LIKE '%select%'

could match ordinary text containing the word select, even when it is not SQL injection or a SQL statement.

Performance risk

Greedy wildcards can consume large amounts of data.

Patterns containing multiple ambiguous wildcards can result in significant backtracking in traditional backtracking regex engines.

Maintainability risk

`.*` gives almost no information about the expected data format.

Compare:

`\d{4}-\d{2}-\d{2}`

with:

`.*`

The first clearly describes a date structure. The second does not define any meaningful constraint.

Conclusion

`.*` should be used only when broad matching is genuinely intended. When the input structure is known, explicit patterns should be used for better correctness, performance, and maintainability.

10. Compare the use of regex versus purpose-built parsers (e.g. HTML parsers, CSV readers) for structured data extraction. In what scenarios does regex become the wrong tool, and what signals in the data should prompt you to switch?

Consider recursive structures, escaped delimiters, and nested tags.

Regex vs purpose-built parsers

Regex is useful when data has a relatively simple and predictable structure.

Examples include:

Dates

Phone numbers

Simple IDs

Simple log entries

Postal codes

However, regex becomes the wrong tool when the data contains complex hierarchical or recursive structure.

HTML

HTML can contain:

`<div>`

`<p>Hello <b>world</b></p>`

`</div>`

There are nested elements.

A regex such as:

<.*?>

can identify simple tags, but it does not understand the HTML tree.

An HTML parser can correctly understand:

div
└─ p
└─ b

CSV

Consider:

John,"Chennai, India",25

The comma inside quotes is part of the field.

A simple:

,

split would be incorrect.

A CSV parser understands quoting and escaped delimiters.

Signals to switch to a parser

Use a parser when you encounter:

- Nested tags
- Recursive structures
- Quoted fields
- Escaped delimiters
- Balanced parentheses
- Complex grammars
- Multiple levels of hierarchy

Conclusion

Regex is best for **recognizing patterns**.

Parsers are best for **understanding structured data**.

11. You are given two patterns to validate IPv4 addresses: `\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}` and a more complex range-validating pattern. Evaluate the trade-off between regex simplicity and semantic correctness — when is 'good enough' acceptable in a data pipeline?

Does the simpler pattern accept 999.999.999.999? Does that matter in your pipeline context?

Simple vs range-validating IPv4 regex

Simple pattern:

```
\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}
```

It expects four groups separated by dots.

For:

192.168.1.10

the groups are:

192

168

1

10

All are syntactically acceptable.

However:

999.999.999.999

also matches.

Why?

Because:

```
\d{1,3}
```

only means:

one to three digits.

It does not mean:

a number between 0 and 255.

Range-validating pattern

A more accurate IPv4 regex restricts each octet to 0–255.

Conceptually, an octet must be one of:

0–9

10–99

100–199

200–249

250–255

This produces a much longer regex.

Trade-off

The simple regex is:

- Short
- Easy to understand
- Easy to maintain
- Fast for basic filtering

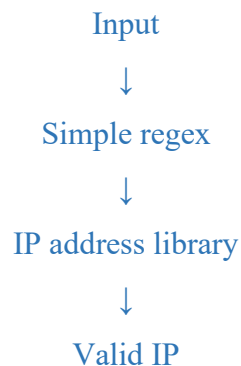
The complex regex is:

- More accurate
- Harder to read
- Harder to maintain
- Better for strict validation

Is accepting 999.999.999.999 important?

It depends on the pipeline.

If the regex is only a preliminary filter:



then the simple regex may be acceptable.

If the regex is the final validation mechanism, accepting 999.999.999.999 is a correctness error.

A good data pipeline should use the simplest validation layer that still satisfies its correctness requirements.

12. Assess the suitability of regex-based tokenisation versus modern NLP tokenisers (e.g. byte-pair encoding) for preprocessing a multilingual query corpus. Under what linguistic conditions does regex fail systematically?

Consider compound words in German, agglutinative languages, and Unicode character classes.

Regex tokenisation vs modern NLP tokenisers

Regex tokenization can work for simple English text but has limitations when processing multilingual corpora.

A simple pattern such as:

`[A-Za-z]+`

assumes that words consist of English alphabetic characters.

This fails for many languages.

German compound words

German allows compound words such as:

Donaudampfschiffahrtsgesellschaft

A regex tokenizer may treat this as one long token.

However, NLP systems may need subword or linguistic analysis depending on the application.

Agglutinative languages

Languages such as Turkish can attach multiple grammatical elements to a single word.

A single surface form can contain information corresponding to multiple English words.

Simple regex tokenization does not understand this morphology.

Unicode

A regex based on:

`[A-Za-z]+`

will not properly capture many scripts.

Examples include:

தமிழ்

العربية

中文

русский

Unicode-aware tokenization is therefore necessary for multilingual corpora.

BPE tokenization

Byte-pair encoding and related subword tokenizers divide words into smaller learned units.

They are useful for:

- Rare words
- Unseen words
- Multilingual text
- Morphologically complex words
- Unicode text

Conclusion

Regex is useful for preprocessing tasks such as:

Remove URLs

Remove mentions

Normalize punctuation

but modern NLP tokenizers are generally more appropriate for actual multilingual tokenization.

13. Design a regex-based data validation layer for a healthcare query system that must parse ICD-10 codes, phone numbers, and patient IDs. How would you organise these patterns for maintainability, and how does your design change if the system needs to run at 10,000 queries per second?

Think about precompiled patterns, DFA vs NFA engines, and separation of validation logic.

Regex validation layer for healthcare queries

A healthcare query system may need to validate:

1. ICD-10 codes
2. Phone numbers
3. Patient IDs

The patterns should be separated according to their purpose.

For example:

```
import re
```

```
ICD10_PATTERN = re.compile(r'...')
```

```
PHONE_PATTERN = re.compile(r'...')
```

```
PATIENT_ID_PATTERN = re.compile(r'...')
```

Separate functions can then be used:

```
def validate_icd10(value):  
    return bool(ICD10_PATTERN.fullmatch(value))
```

```
def validate_phone(value):  
    return bool(PHONE_PATTERN.fullmatch(value))
```

```
def validate_patient_id(value):  
    return bool(PATIENT_ID_PATTERN.fullmatch(value))
```

Why this design is maintainable

Each validation rule is independent.

Therefore:

- A change to phone validation does not affect ICD-10 validation.
- Each pattern can have separate test cases.
- Errors can be reported accurately.
- Patterns can be reused across the application.

At 10,000 queries per second

Performance becomes important.

1. Precompile patterns

Instead of repeatedly compiling:

```
re.compile(...)
```

compile once during application startup.

2. Use fullmatch()

For validation, use:

```
PATTERN.fullmatch(value)
```

rather than searching through the entire string.

3. Avoid catastrophic backtracking

Do not use unnecessarily ambiguous patterns such as:

```
(a+)+
```

4. Separate validation from business logic

The regex layer should answer:

Does this input have the expected format?

Business logic should answer:

Is this value valid according to healthcare rules?

For example, a syntactically correct ICD-10 code still needs to be checked against an authoritative code set if actual medical-code validity is required.

DFA vs NFA

Backtracking NFA engines can be very fast for many patterns but can have worst-case performance problems.

DFA or linear-time engines provide more predictable performance for suitable regular expressions.

For a high-throughput system, predictable execution time is important.

14. Build a framework for automatically generating regex patterns from a set of positive and negative example strings. What algorithmic challenges arise, and how would you evaluate the precision and recall of the generated patterns on held-out test data?

This connects to the field of 'regex synthesis' — think about over-fitting to examples.

Framework for automatically generating regex patterns

Automatically generating regexes from examples is known as **regex synthesis**.

Suppose the positive examples are:

AB123

CD456

XY789

and negative examples are:

ABC123

12345

AB12

The desired pattern could be:

`[A-Z]{2}\d{3}`

Framework

Step 1: Collect examples

Separate the dataset into:

Positive examples

Negative examples

Step 2: Extract common features

Look for:

- Character types
- String length
- Repeated structures
- Fixed prefixes
- Fixed suffixes
- Optional components

The examples:

AB123

CD456

XY789

suggest:

2 uppercase letters + 3 digits

Step 3: Generate candidate regexes

Candidate patterns might contain:

Character classes

Quantifiers

Alternation

Optional components

Anchors

Literals

Step 4: Test candidates

A candidate is tested against positive and negative examples.

An ideal pattern should:

Match positive examples

Reject negative examples

Overfitting problem

The system could generate:

AB123|CD456|XY789

This matches all training examples, but it has simply memorized them.

A more general pattern:

`[A-Z]{2}\d{3}`

is better if it also correctly handles unseen examples.

Evaluation

Use separate training and test datasets.

Measure:

$\text{Precision} = \frac{TP}{TP + FP}$ $\text{Recall} = \frac{TP}{TP + FN}$ $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Also measure:

- Pattern complexity
- Regex length
- Execution time
- False positives
- False negatives

Main challenge

The objective is not simply to find a regex that fits the examples.

The objective is to find the **simplest regex that generalizes correctly to unseen data**.

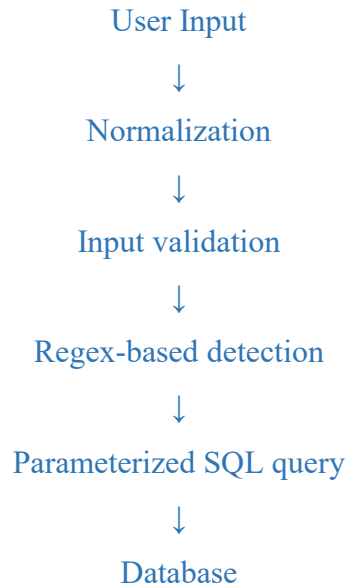
15. You are designing a query rewriting engine that uses regex to detect and transform SQL injection patterns in user inputs before they reach the database. Synthesise a strategy that balances security coverage, false positive rate, and query latency. What are the limits of a regex-only approach?

Consider parameterised queries as the ground truth — where does regex complement rather than replace them?

Regex-based SQL injection detection strategy

Regex can be used to detect suspicious SQL injection patterns, but it should not be the primary security mechanism.

A layered architecture is recommended:



Step 1 – Normalize input

Normalize obvious variations such as:

- Case
- Encoding
- Whitespace

This can make detection more consistent.

Step 2 – Apply regex detection

Regex can flag suspicious patterns involving SQL keywords, comments, or unexpected query syntax.

The purpose is mainly:

Detection

Logging

Alerting

Additional filtering

Step 3 – Use parameterized queries

The primary defense should be parameterized queries.

Instead of constructing SQL by concatenating input:

`SELECT * FROM users WHERE name = 'USER_INPUT'`

use:

SELECT * FROM users WHERE name = ?

and pass the user input separately.

The database then treats the input as data instead of SQL syntax.

Why regex alone is insufficient

An attacker can modify the input using:

- Case changes
- Whitespace
- Comments
- Encoding
- Alternative SQL syntax
- Database-specific syntax

A regex may therefore:

- Miss an attack → false negative
- Block legitimate input → false positive

Security and performance balance

Requirement	Approach
Security	Parameterized queries
Additional detection	Regex
False positives	Use narrow patterns
Low latency	Precompile regexes
Avoid DoS	Avoid catastrophic backtracking
Monitoring	Log suspicious inputs

Final conclusion

Regex should **complement**, not replace, parameterized queries.

The correct security model is:

Regex = detection / filtering / monitoring

Parameterized queries = primary SQL injection defense

Database security controls = additional protection

A regex-only SQL injection prevention system cannot guarantee security because SQL has a complex grammar and attackers can construct equivalent queries in many different ways.